

System 14: In-Memory Pub/Sub Message Broker (Kafka Lite)

Zero-Prerequisite Intuition: The "Newspaper Subscription" Metaphor What is a Publish/Subscribe (Pub/Sub) message broker, and why did the software industry invent it? Imagine a news agency that prints financial news. If the news agency had to personally call 50,000 individual readers on the telephone every time a story broke, the journalists would spend 100% of their day dialing phones. If 5,000 new readers sign up, the journalists are completely overwhelmed.

Instead, the agency prints newspapers and drops them into a central **Newsstand (The Topic)**.

- **Publishers (Journalists):** Drop articles into the newsstand and immediately walk away.
- **Subscribers (Readers):** Walk up to the newsstand whenever they are free, read the articles at their own speed, and remember which page they finished reading (**The Offset**).

In microservices (Amazon, Netflix, Uber), Order Services don't call Billing, Shipping, Analytics, and Fraud directly. They publish an `OrderPlaced` event to a message broker, and each downstream team consumes it independently!

This chapter designs an **In-Memory Pub/Sub Message Broker** (a simplified Apache Kafka engine) in pure Python.

1. Requirements & Core Concepts

Core Primitives Defined

1. **Message:** An immutable payload with a `key`, `value`, `timestamp`, and assigned `offset`.
2. **Topic:** A named logical stream of messages (e.g., `user-signups`).
3. **Partition:** A strictly ordered, append-only log within a topic that guarantees message order.
4. **Consumer Group:** A collection of consumers cooperating to read data. Each message sent to a topic is delivered to **one consumer instance within each subscribing consumer group**.

```
graph TD
    P1["Producer A"] --> T_P0["Topic 'orders' - Partition 0"]
    P2["Producer B"] --> T_P1["Topic 'orders' - Partition 1"]

    subgraph Consumer_Group_Billing ["Consumer Group: 'billing-team'"]
        C1["Consumer 1 (Reads P0)"]
        C2["Consumer 2 (Reads P1)"]
    end

    subgraph Consumer_Group_Analytics ["Consumer Group: 'analytics-team'"]
        C3["Consumer 3 (Reads P0 & P1)"]
    end

    T_P0 --> C1
    T_P1 --> C2
    T_P0 --> C3
    T_P1 --> C3
```

2. Design Pattern & Class Hierarchy

```

classDiagram
    direction TB
    class Message {
        +int offset
        +str key
        +str payload
        +float timestamp
    }
    class Partition {
        -int partition_id
        -list[Message] log
        -Lock lock
        +append(key, payload) Message
        +read_from(offset, limit) list[Message]
    }
    class Topic {
        +str name
        -list[Partition] partitions
        +publish(key, payload) Message
        +get_partition(id) Partition
    }
    class ConsumerGroup {
        +str group_id
        -dict[int, int] partition_offsets
        +commit(partition_id, offset) void
        +get_offset(partition_id) int
    }
    class Broker {
        -dict[str, Topic] topics
        -dict[str, ConsumerGroup] groups
        +create_topic(name, num_partitions) void
        +publish(topic, key, val) void
        +consume(topic, group_id, partition_id, limit) list[Message]
    }

    Topic o--> Partition
    Partition o--> Message
    Broker o--> Topic
    Broker o--> ConsumerGroup
  
```


3. Complete Python Implementation

```
# pub_sub_broker.py
import threading
import time
from typing import Dict, List, Optional
from dataclasses import dataclass

# =====
# 1. Immutable Message
# =====

@dataclass(frozen=True)
class Message:
    offset: int
    key: Optional[str]
    payload: str
    timestamp: float

# =====
# 2. Append-Only Partition Log
# =====

class Partition:
    def __init__(self, partition_id: int):
        self.partition_id = partition_id
        self._log: List[Message] = []
        self._lock = threading.Lock()

    def append(self, key: Optional[str], payload: str) → Message:
        with self._lock:
            offset = len(self._log)
            msg = Message(
                offset=offset,
                key=key,
                payload=payload,
                timestamp=time.time()
            )
            self._log.append(msg)
            return msg

    def read_from(self, offset: int, max_messages: int = 10) → List[Message]:
        with self._lock:
```

```

        if offset ≥ len(self._log):
            return []
        return self._log[offset : offset + max_messages]

# =====
# 3. Partitioned Topic
# =====

class Topic:
    def __init__(self, name: str, num_partitions: int = 3):
        self.name = name
        self.num_partitions = num_partitions
        self.partitions: List[Partition] = [Partition(i) for i in range(num_partitions)]

    def _get_partition_index(self, key: Optional[str]) → int:
        if key is None:
            # Round-robin or default to partition 0
            return 0
        # Deterministic hashing: same key always maps to same partition!
        return hash(key) % self.num_partitions

    def publish(self, key: Optional[str], payload: str) → Message:
        p_idx = self._get_partition_index(key)
        partition = self.partitions[p_idx]
        return partition.append(key, payload)

# =====
# 4. Consumer Group & Offset State
# =====

class ConsumerGroup:
    def __init__(self, group_id: str):
        self.group_id = group_id
        # Tracks current committed offset: partition_id → next_offset_to_read
        self.offsets: Dict[int, int] = {}
        self._lock = threading.Lock()

    def get_offset(self, partition_id: int) → int:
        with self._lock:
            return self.offsets.get(partition_id, 0)

    def commit_offset(self, partition_id: int, processed_offset: int) → None:
        with self._lock:
            # Next read begins at processed_offset + 1

```

```

        self.offsets[partition_id] = processed_offset + 1

# =====
# 5. Master In-Memory Broker
# =====

class InMemoryBroker:
    def __init__(self):
        self.topics: Dict[str, Topic] = {}
        self.consumer_groups: Dict[str, ConsumerGroup] = {}
        self._lock = threading.Lock()

    def create_topic(self, name: str, num_partitions: int = 3) → Topic:
        with self._lock:
            if name in self.topics:
                raise ValueError(f"Topic '{name}' already exists.")
            topic = Topic(name, num_partitions)
            self.topics[name] = topic
            return topic

    def publish(self, topic_name: str, payload: str, key: Optional[str] = None) → Message:
        topic = self.topics.get(topic_name)
        if not topic:
            raise KeyError(f"Topic '{topic_name}' does not exist.")
        return topic.publish(key, payload)

    def consume(self, topic_name: str, group_id: str, partition_id: int, limit: int = 5) → List[Message]:
        topic = self.topics.get(topic_name)
        if not topic:
            raise KeyError(f"Topic '{topic_name}' does not exist.")

        with self._lock:
            if group_id not in self.consumer_groups:
                self.consumer_groups[group_id] = ConsumerGroup(group_id)
            group = self.consumer_groups[group_id]

        current_offset = group.get_offset(partition_id)
        partition = topic.partitions[partition_id]

        messages = partition.read_from(current_offset, max_messages=limit)
        if messages:
            # Advance offset to the last read message
            last_offset = messages[-1].offset

```

```
group.commit_offset(partition_id, last_offset)

return messages
```

4. Verification: Multi-Consumer Group Isolation

```
if __name__ == "__main__":
    broker = InMemoryBroker()
    broker.create_topic("payments.v1", num_partitions=2)

    # 1. Publish events with keys
    msg1 = broker.publish("payments.v1", payload="Charge $100 for User Alice", key="user_alice")
    msg2 = broker.publish("payments.v1", payload="Charge $50 for User Bob", key="user_bob")
    msg3 = broker.publish("payments.v1", payload="Charge $200 for User Alice", key="user_alice")

    print(f"Published 3 messages. Notice how user_alice events landed on same partition!")

    # 2. Consumer Group 1: Fraud Detection
    fraud_batch = broker.consume("payments.v1", group_id="fraud-service", partition_id=0)
    print(f"\n[Fraud Service] Read {len(fraud_batch)} events from Partition 0:")
    for m in fraud_batch:
        print(f"  Offset {m.offset}: {m.payload}")

    # 3. Consumer Group 2: Analytics (Completely independent offsets!)
    analytics_batch = broker.consume("payments.v1", group_id="analytics-service", partition_id=0)
    print(f"\n[Analytics Service] Read {len(analytics_batch)} events from Partition 0:")
    for m in analytics_batch:
        print(f"  Offset {m.offset}: {m.payload}")
```

Both consumer groups read the exact same data without interfering with each other's reading progress!